

Trees

- 1-a: Introduction of Tree Data Structure/ADT - Tree Terminology
- 1-b: Applications of tree data structure
- 2. Types of Trees
- 3. Types of Binary Trees
- 4. Binary Tree Properties
- 5. Binary Tree Traversal
 - 5.a: Breadth First Search , BFS
 - 5.b: Depth First Search, DFS: Tree Traversals (Inorder, Preorder and Postorder)
- 6-a: Expression Tree
- 6-b: Evaluation of Expression Tree ? **For you...**

Trees

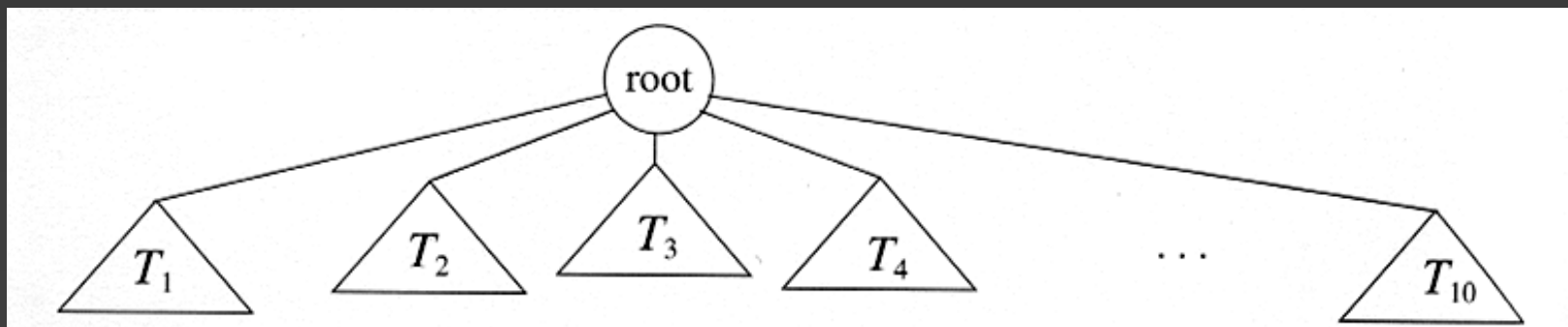
✉ Linear access time of linked lists is prohibitive

Does there exist any simple data structure for which the running time of most operations (search, insert, delete) is $O(\log N)$?

Trees

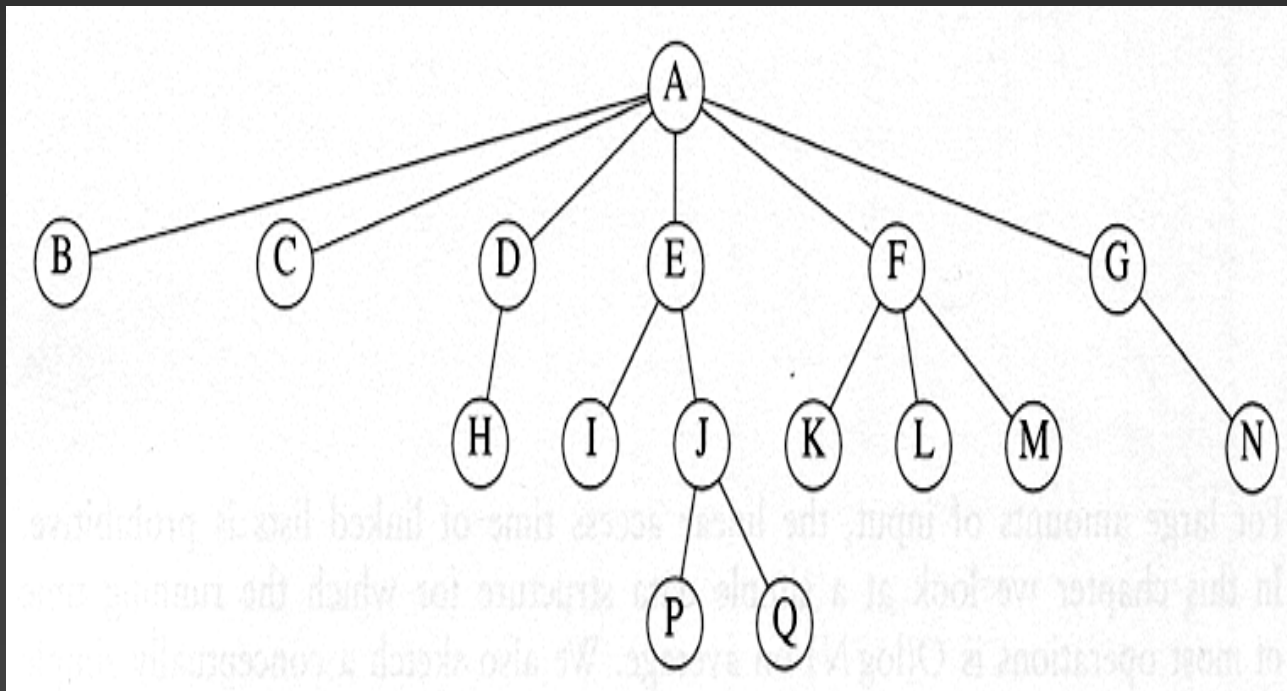


- ✉ A tree is a collection of nodes
- The collection can be empty
 - (recursive definition) If not empty, a tree consists of a distinguished node r (the **root**), and **zero or more** nonempty **subtrees** $T_1, T_2, \dots, T_k$, each of whose roots are connected by a directed **edge** from r



Trees

✉ A tree is a collection of nodes



Example: UNIX Directory

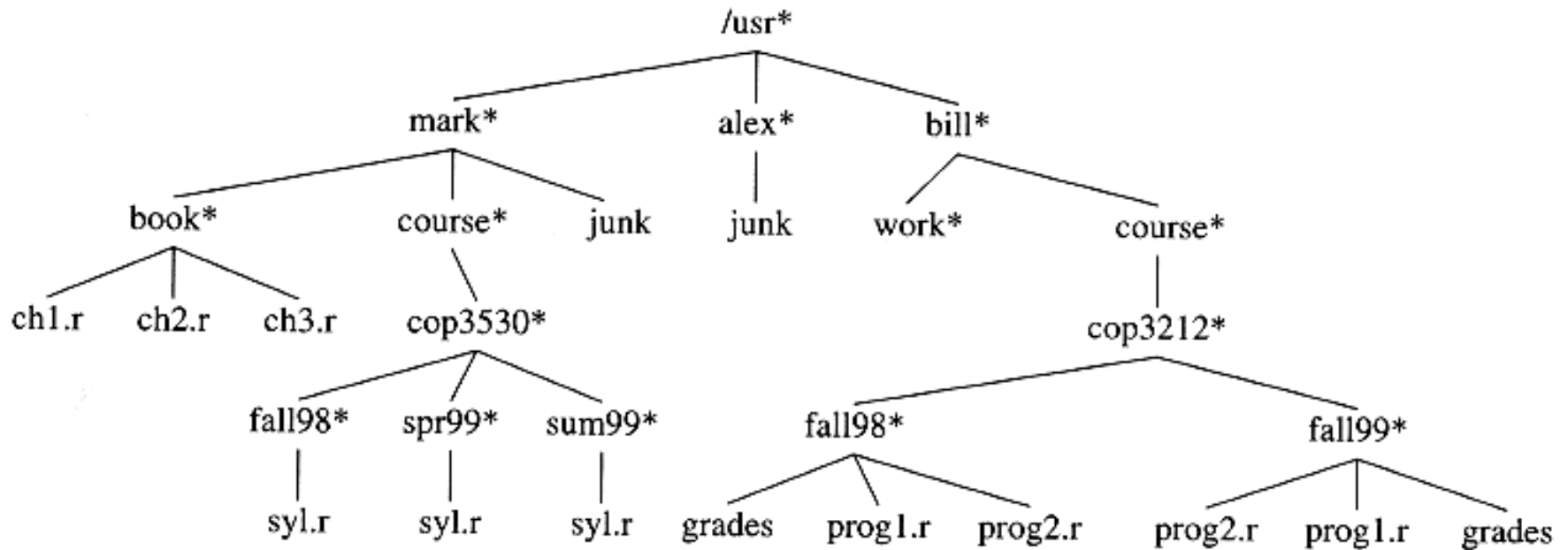
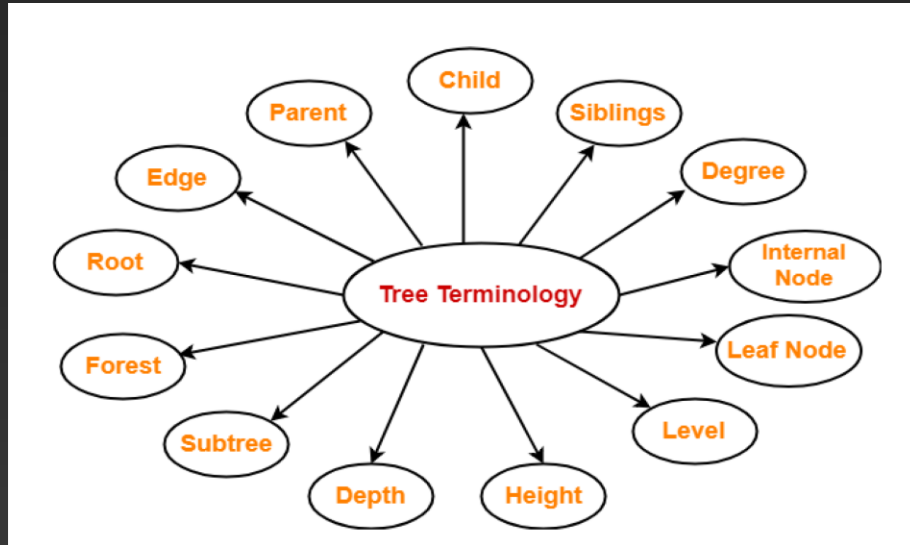


Figure 4.5 UNIX directory

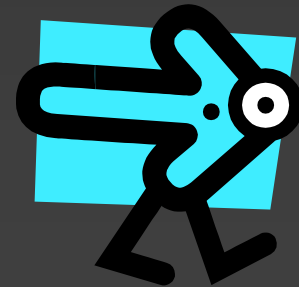
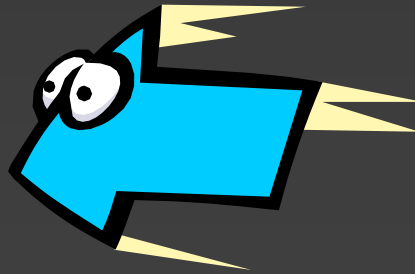
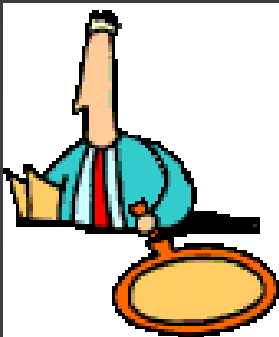
Some Terminologies



- **Child and Parent**
 - Every node except the root has one parent
 - A node can have an zero or more children
- **Leaves**
 - Leaves are nodes with no children
- **Sibling**
 - nodes with same parent
- **Path**
 - A sequence of edges
- **Length of a path**
 - number of edges on the path
- **Depth of a node**
 - length of the unique path from the root to that node
- **Height of a node**
 - length of the longest path from that node to a leaf
 - all leaves are at height 0
- **The height of a tree = the height of the root = the depth of the deepest leaf**
- **Ancestor and descendant**
 - If there is a path from n_1 to n_2
 - n_1 is an ancestor of n_2 , n_2 is a descendant of n_1
 - *Proper ancestor* and *proper descendant*
- **The others....**

Tree Traversal

- ✉ Used to print out the data in a tree in a certain order
- ✉ **Pre-order traversal**
 - Print the data at the **root**
 - Recursively print out all data in the **left subtree**
 - Recursively print out all data in the **right subtree**



Preorder, Postorder and Inorder

✉ Preorder traversal

- node, left, right
- prefix expression

✉ ++a*bc*+*defg

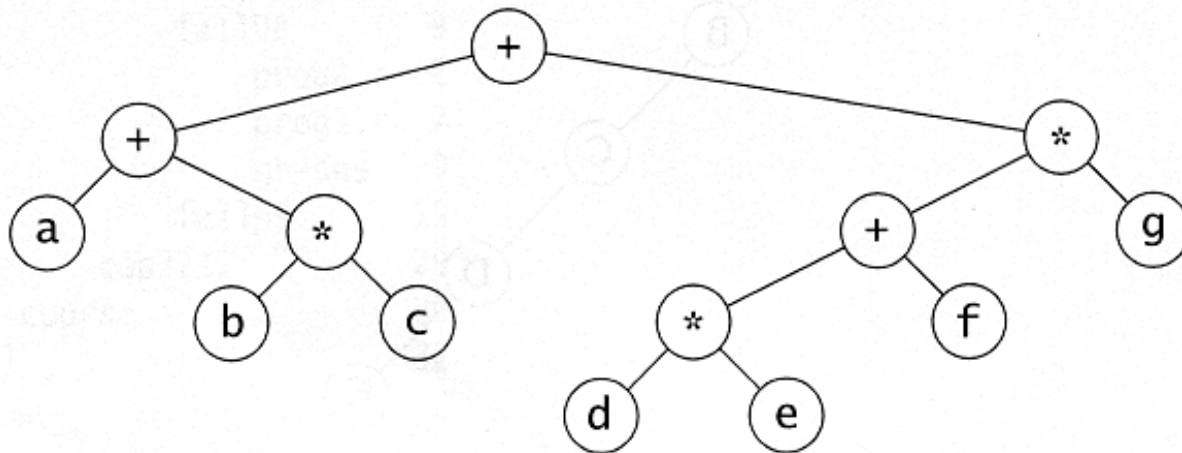


Figure 4.14 Expression tree for $(a + b * c) + ((d * e + f) * g)$

Preorder, Postorder and Inorder

✉ Postorder traversal

- left, right, node
- postfix expression

abc*+de*f+g*+

✉ Inorder traversal

- left, node, right
- infix expression

a+b*c+d*e+f*g

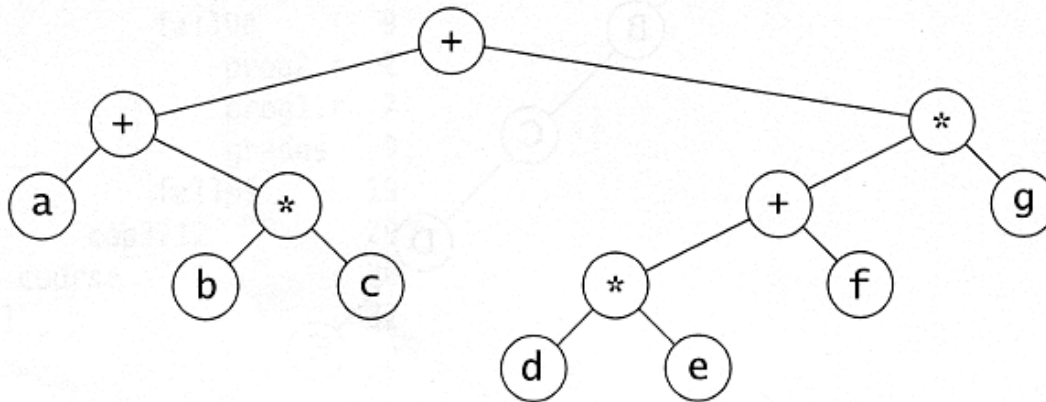


Figure 4.14 Expression tree for $(a + b * c) + ((d * e + f) * g)$

Example: Unix Directory Traversal

PreOrder

```
/usr
  mark
    book
      ch1.r
      ch2.r
      ch3.r
    course
      cop3530
        fall98
          syl.r
        spr99
          syl.r
        sum99
          syl.r
      junk
    alex
      junk
    bill
      work
      course
        cop3212
          fall98
            grades
            prog1.r
            prog2.r
          fall99
            prog2.r
            prog1.r
            grades
```

PostOrder

```
  ch1.r      3
  ch2.r      2
  ch3.r      4
  book      10
    syl.r    1
  fall98    2
    syl.r    5
  spr99     6
    syl.r    2
  sum99     3
  cop3530   12
  course    13
  junk      6
  mark      30
  junk      8
  alex      9
  work      1
    grades   3
    prog1.r  4
    prog2.r  1
  fall98    9
    prog2.r  2
    prog1.r  7
    grades   9
  fall99    19
  cop3212   29
  course    30
  bill      32
/usr       72
```

Preorder, Postorder and Inorder Pseudo Code

Algorithm *Preorder*(x)

Input: x is the root of a subtree.

1. **if** $x \neq \text{NULL}$
2. **then** output key(x);
3. *Preorder*(left(x));
4. *Preorder*(right(x));

Algorithm *Postorder*(x)

Input: x is the root of a subtree.

1. **if** $x \neq \text{NULL}$
2. **then** *Postorder*(left(x));
3. *Postorder*(right(x));
4. output key(x);

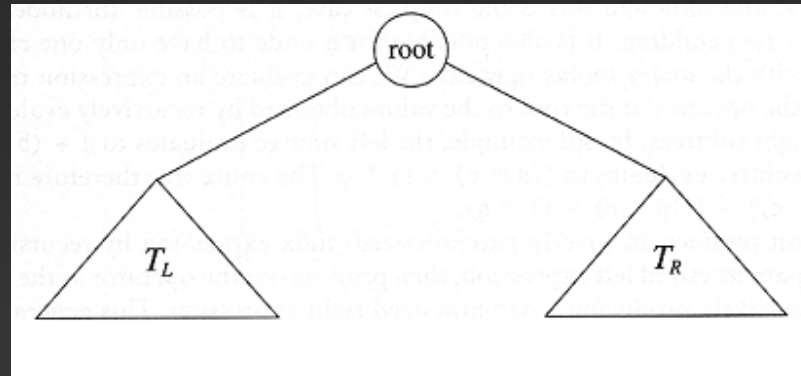
Algorithm *Inorder*(x)

Input: x is the root of a subtree.

1. **if** $x \neq \text{NULL}$
2. **then** *Inorder*(left(x));
3. output key(x);
4. *Inorder*(right(x));

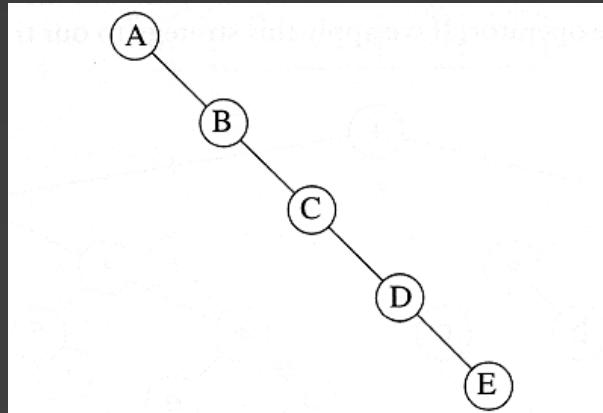
Binary Trees

- ✉ A tree in which **no node can have more than two children**



**Generic
binary tree**

- ✉ The depth of an “average” binary tree is considerably smaller than N , even though in the worst case, the depth can be as large as $N - 1$.



**Worst-case
binary tree**

Node Struct of Binary Tree

✉ Possible operations on the Binary Tree ADT

- Parent, left_child, right_child, sibling, root, etc

✉ Implementation

- Because a binary tree has at most two children, we can keep direct pointers to them

```
struct BinaryNode
{
    Object      element;           // The data in the node
    BinaryNode *left;              // Left child
    BinaryNode *right;             // Right child
};
```

Convert a Generic Tree to a Binary Tree

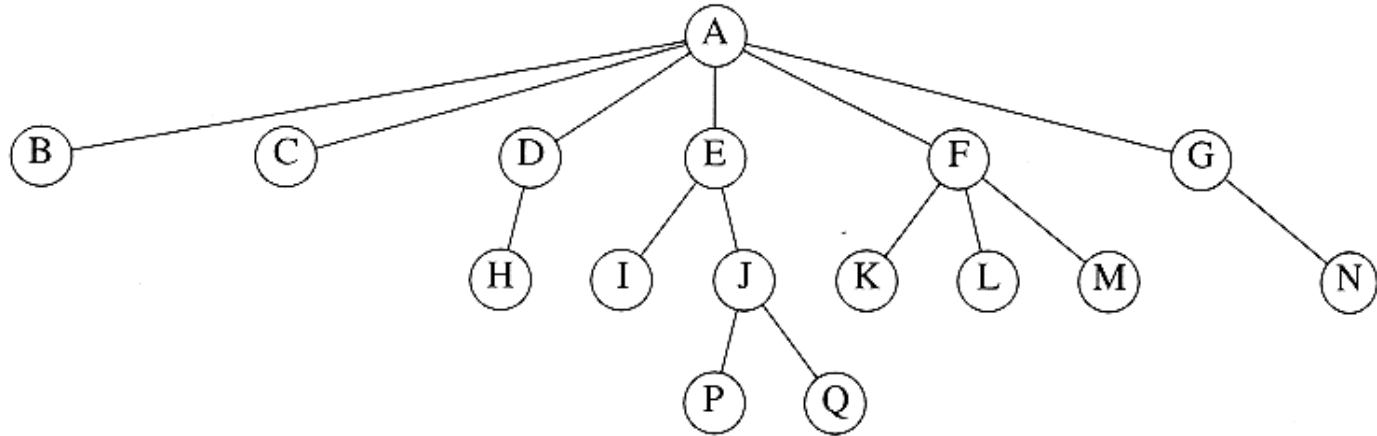


Figure 4.2 A tree

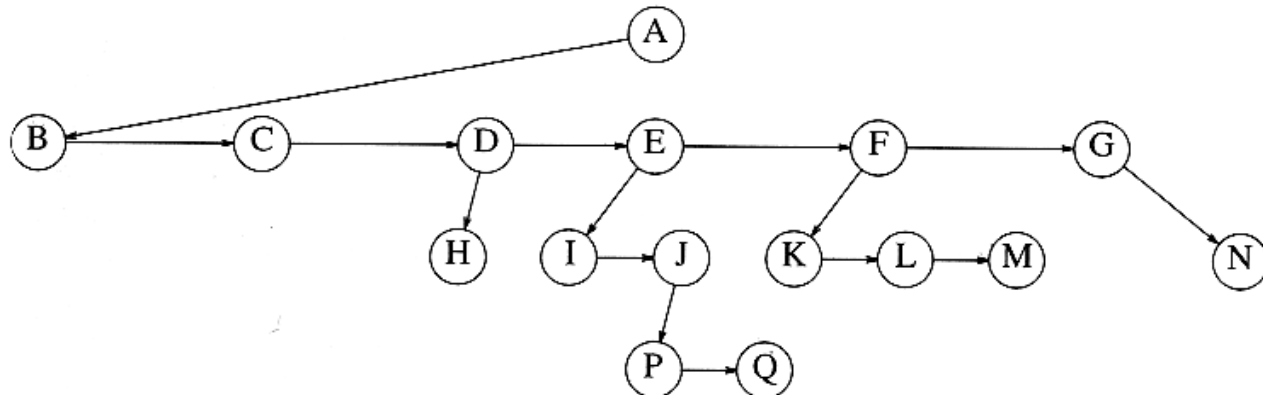


Figure 4.4 First child/next sibling representation of the tree shown in Figure 4.2

Example: Expression Trees

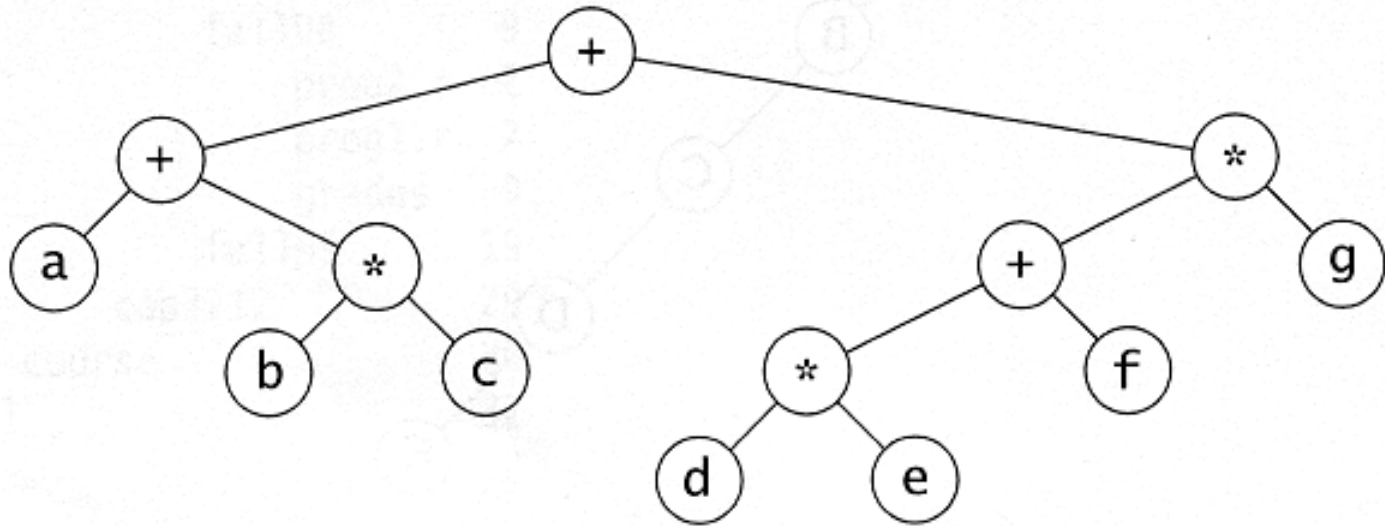


Figure 4.14 Expression tree for $(a + b * c) + ((d * e + f) * g)$

- ✉ **Leaves** are **operands** (constants or variables)
- ✉ The **internal nodes** contain **operators**
- ✉ Will not be a binary tree if some operators are not binary